Informe de la Etapa 4: Índices y Rendimiento · Bodegas del Atuel

1. Medición baseline (#31)

Esta medición es el punto de partida contra el que se comparan los índices de `008_indices.sql`. Se hizo solo con los índices de claves primarias y UNIQUE: la base no tiene ningún índice no único.

1.1 Entorno y método

Dato	Valor
Motor	PostgreSQL 16.15 en contenedor `postgres:16`, configuración por defecto
Migraciones aplicadas	`001_esquema.sql` a `007_roles.sql`
Datos	`seed.py --seed 42 --fecha-ref 2026-11-05` (729.551 filas, C01..C14 sin inconsistencias)
Estadísticas	`VACUUM ANALYZE` antes de medir
Consultas	consultas_medicion.sql, con `EXPLAIN (ANALYZE, BUFFERS)`
Corridas	6: la primera calienta la caché y se informa la mediana de las otras 5

`--fecha-ref` coincide con la fecha de la medición porque S3, S4 y S8 calculan su ventana con `CURRENT_DATE`. A las candidatas sugeridas por el enunciado (S1, S4, S8 y S10) se agregaron S3, porque filtra una tabla grande por dos columnas y devuelve pocas filas; S5, porque casi todas las transferencias ya están procesadas; y la condición de D4, medida como `SELECT`, porque depende de una clave foránea sin índice.

Parámetros usados: sala 12 en S3 y vino 1 («Malbec Reserva B1», 2020, 750 ml) en S4.

La decisión de comparar S10 dentro de cada campaña está explicada en el encabezado de la consulta.

1.2 Resultados

Consulta	Tiempo (mediana)	Recorrido principal	Filas leídas	Filas útiles	Buffers
S1	241,3 ms	Parallel Seq Scan en `renglon_comprobante`	500.000	500.000	3.745
S3	9,2 ms	Parallel Seq Scan en `comprobante`	150.000	835	2.938
S4	1,1 ms	Seq Scan en `renglon transferencia` y `transferencia`	10.000 y 2.500	327 y 186	69
S5	0,4 ms	Seq Scan en `transferencia`	2.500	150	15
S8	26,0 ms	Parallel Seq Scan en `comprobante`	150.000	42.115	1.260
S10	531,7 ms	CTE Scan repetido sobre `rendimiento`	10.562.500	16 por recorrido del CTE, en promedio	36
D4 (condición)	112,2 ms	Seq Scan en `comprobante` y `cliente`	150.000 y 20.000	2.773 en el resultado	1.535

Los planes completos y el tiempo de cada corrida están en `anexo_planes_baseline.txt` .

1.3 Qué muestran los planes

- **S1:** lee toda la tabla porque agrega todos los renglones. No hay filas que un índice pueda descartar, así que la mejora dependerá de un recorrido solo de índice o de evitar el ordenamiento.
- **S3:** es el caso más claro de filtro sin índice. Lee 150.000 filas para quedarse con 835, un 0,6 %.
- **S4:** la PK `(nro, cod_vino)` no sirve para buscar por `cod_vino` , porque esa no es su primera columna. Las tablas son chicas, así que la mejora en tiempo será pequeña.
- **S8:** conserva un 28 % de las filas. Esa selectividad es baja para un índice común, por lo que un índice solo sobre `fecha` podría no usarse.
- **S10:** es la más lenta, pero el índice no es la solución. Referenciar el CTE dentro de la subconsulta correlacionada lo recorre entero 3.250 veces (3.250 × 3.250 = 10.562.500 filas), y un CTE materializado no tiene índices. Para mejorarla hay que reescribir la consulta.
- **S5:** es muy rápida porque la tabla es chica, pero lee 2.500 filas para usar 150.
- **D4 (condición):** para saber qué socios no compraron, recorre todo `comprobante` , porque su clave foránea `id_cliente` no tiene índice.

2. Índices creados (#32)

Los índices están en `008_indices.sql` . Se probaron dentro de transacciones con `ROLLBACK` , para no alterar el baseline, y quedaron solo los que mostraron una mejora.

Índice	Tipo pedido	Definición	Consulta	Tamaño
`idx_comprobante_sala_fecha`	Compuesto	`comprobante (id_sala, fecha)`	S3	2.040 kB
`idx_comprobante_cliente`	Clave foránea	`comprobante (id_cliente)`	D4	1.064 kB
`idx_transferencia_pendientes`	Parcial	`transferencia (id sala) WHERE estado <> 'procesado'`	S5	16 kB
`idx_renglon_transferencia_vino`	Variante `INCLUDE`	`renglon transferencia (cod_vino) INCLUDE (nro, cantidad)`	S4	328 kB
`idx_transferencia_fecha`	Simple	`transferencia (fecha_confeccion)`	S4	40 kB

Orden de columnas del índice compuesto: primero `id_sala` , que S3 compara por igualdad, y después `fecha` , que compara por rango. Con ese orden, el índice llega a una sola sala y recorre solo su tramo de fechas. Con `(fecha, id_sala)` recorrería todas las salas dentro del rango de fechas.

Los tamaños corresponden a la carga de la sección 1.1. Dependen de los datos: con `--fecha-ref 2026-11-03` , el índice compuesto medía 1.632 kB, porque había 26.254 pares distintos de sala y fecha, contra 44.336 en esta carga.

2.1 Candidatos descartados

Estas pruebas también se hicieron dentro de transacciones con `ROLLBACK` , con la mediana de 5 corridas salvo donde se indica. Se hicieron sobre la carga anterior, con `--fecha-ref 2026-11-03` , por lo

que sus tiempos no se comparan directamente con los de la sección 1.2.

Candidato	Consulta	Resultado	Por qué se descartó
`comprobante (fecha) INCLUDE (id_sala, total)`	S8	Baja de 1.257 a 165 buffers, pero sube de 25,6 a 32,3 ms	Ocupa 4.640 kB y empeora el tiempo
`renglon comprobante (cod vino) INCLUDE (cantidad)`	S1	El planificador no lo usó (una corrida)	S1 agrega toda la tabla y el índice ocupa 11 MB
BRIN sobre `comprobante (fecha)`	S8	No se creó: la correlación física de `fecha` es 0,025	BRIN necesita filas ordenadas físicamente por esa columna
Índice para S10	S10	No hay candidato	El tiempo se va en recorrer el CTE, que no admite índices

El índice de S8 pierde porque el plan original usa dos procesos en paralelo y el Index Only Scan corre en uno solo. Sin paralelismo (`max_parallel_workers_per_gather = 0`), el mismo índice sí gana: baja de 42,2 a 31,4 ms. Conviene volver a evaluarlo si el servidor en la nube tiene un solo núcleo.

3. Comparativa, costo de escritura y caso sin uso de índice (#33)

3.1 Tabla comparativa

Las dos mediciones usan la misma carga, el mismo script y el mismo día. Solo cambia que se aplicó `008_indices.sql` , seguido de `VACUUM ANALYZE` . Los planes están en `anexo_planes_baseline.txt` y `anexo_planes_con_indices.txt` .

Consulta	Antes	Después	Cambio	Recorrido antes	Recorrido después	Filas leídas antes → después	Buffers
S3	9,2 ms	5,7 ms	-37 %	Parallel Seq Scan	Bitmap Index Scan	150.000 → 835	2.938 → 1.237
S4	1,1 ms	0,6 ms	-44 %	Seq Scan en ambas tablas	Index Only Scan y Bitmap Index Scan	10.000 → 327 y 2.500 → 186	69 → 21
S5	0,4 ms	0,3 ms	-22 %	Seq Scan	Bitmap Index Scan	2.500 → 150	15 → 4
D4 (condición)	112,2 ms	10,9 ms	-90 %	Seq Scan y Hash Anti Join	Index Only Scan y Nested Loop Anti Join	Recorre `comprobante` entero → busca por cliente	1.535 → 7.236
S1	241,3 ms	246,2 ms	Sin mejora	Parallel Seq Scan	Igual	500.000 → 500.000	3.745 → 3.745
S8	26,0 ms	27,2 ms	Sin mejora	Parallel Seq Scan	Igual	150.000 → 150.000	1.260 → 1.260
S10	531,7 ms	523,2 ms	Sin mejora	CTE Scan	Igual	10.562.500 → 10.562.500	36 → 36

Hay mejora medida en cuatro consultas. En S1, S8 y S10 el plan no cambió, así que esas diferencias de tiempo son variación entre corridas.

En D4 los buffers suben aunque el tiempo baja mucho. El Nested Loop busca en el índice una vez por cada uno de los 3.475 socios candidatos, y cuenta varias veces las mismas páginas, que ya están en memoria. Recorrer la tabla entera y armar la tabla de hash cuesta más tiempo que esas búsquedas.

S1, S8 y S10 no mejoran por motivos distintos: S1 necesita toda la tabla, S8 conserva el 28 % de las filas y S10 pasa su tiempo en un CTE. El índice que se probó para S8 está en la sección 2.1.

3.2 Costo de escritura

Se usó la carga real: `seed.py` sobre dos bases recién migradas, una con 001 a 007 y otra con 001 a 008. Se alternaron 6 cargas por base, se descartó la primera de cada una y se informa la mediana de las otras 5. Ambas bases terminaron con los mismos datos. `seed.py` informa el tiempo de `COPY` de cada tabla.

Tabla	Índices nuevos	Sin `008`	Con `008`	Diferencia
`comprobante` (150.000 filas)	2	1,368 s	1,611 s	+0,243 s (+18 %)
`renglon_transferencia` (10.000 filas)	1	0,083 s	0,093 s	+0,010 s (+12 %)
`transferencia` (2.500 filas)	2	0,015 s	0,018 s	+0,003 s (+20 %)
`renglon_comprobante` (500.000 filas), control	0	4,288 s	4,256 s	-0,032 s (-1 %)
Carga completa (729.551 filas)	5	7,25 s	7,50 s	+0,25 s (+3 %)

`renglon_comprobante` funciona como control: no recibe índices nuevos y su tiempo prácticamente no cambia.

El costo en disco es de 3,4 MB si los índices se crean después de la carga y de 4,0 MB si ya existen durante la carga. Por eso conviene crearlos después cuando se carga una base vacía.

El costo es aceptable: la carga completa tarda un 3 % más, y las consultas de la sección 3.1 mejoran hasta un 90 %.

3.3 Caso en que el planificador no usa un índice

El script es `caso_sin_uso_indice.sql`, y usa `idx_comprobante_sala_fecha`, cuya primera columna es `id_sala`.

Caso	Filtro	Filas	Plan	Tiempo (mediana)
A	`id_sala BETWEEN 1 AND 30`	125.004 (83 %)	Parallel Seq Scan: no usa el índice	54,0 ms
B	El mismo, con `enable_seqscan = off`	125.004 (83 %)	Bitmap Index Scan: usa el índice forzado	94,3 ms
C	`id_sala = 12`	4.167 (2,8 %)	Bitmap Index Scan: usa el índice	4,4 ms

Causa: baja selectividad. En el caso A hay que leer el 83 % de las filas y, de todas formas, casi todas las páginas de la tabla. Pasar por el índice agrega trabajo sin evitar lecturas. El caso B muestra que el planificador acertó: al forzar el índice, la consulta tarda un 75 % más. El caso C muestra que el índice funciona cuando el filtro es selectivo.

También S8 ignora `idx_comprobante_sala_fecha`, por otro motivo: filtra solo por `fecha`, que es la segunda columna del índice.